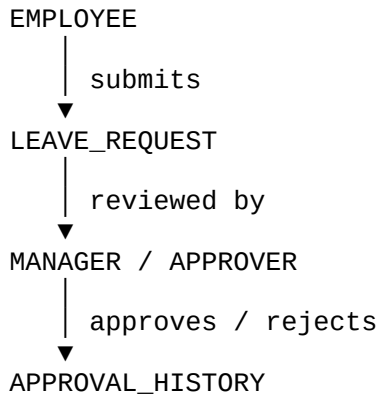


Rohit Raj Singh

day 10

Correct RBAC relationship



DAY 10 – Database Design & Business Rules

Employee Leave Management System

1. Objective

Design a relational database for an Employee Leave Management System that allows employees to apply for leave, maintains leave balances, and allows authorized managers/approvers to approve or reject leave requests.

The design should enforce data integrity and important business rules such as sufficient leave balance, no overlapping leave, and prevention of self-approval.

2. Main Entities

The system contains these core entities:

1. **Department**
2. **Employee**
3. **Role**
4. **Leave Type**
5. **Leave Balance**
6. **Leave Request**
7. **Approval History**

Adding a Role entity makes the design more realistic because RBAC requires us to know whether an employee is an Employee, Manager, HR, or Administrator.

3. Entity Details

3.1 DEPARTMENT

Stores information about company departments.

Column	Data Type	Key / Constraint
department_id	INT	PK
department_name	VARCHAR(100)	NOT NULL, UNIQUE
description	VARCHAR(255)	Optional

Example

1 → Engineering
2 → Human Resources
3 → Finance
4 → Marketing

3.2 ROLE

Defines the role/authorization level of an employee.

Column	Data Type	Key / Constraint
role_id	INT	PK
role_name	VARCHAR(50)	NOT NULL, UNIQUE
description	VARCHAR(255)	Optional

Example roles

1 → Employee
2 → Manager
3 → HR
4 → Admin

The **Role** table is used for RBAC.

For example:

- Employee → can apply for leave
- Manager → can approve/reject subordinate requests
- HR → can manage leave policies/balances
- Admin → system administration

3.3 EMPLOYEE

Stores employee information.

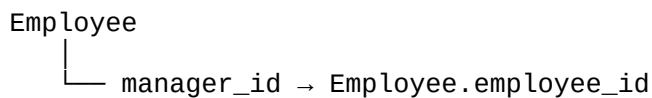
Column	Data Type	Key / Constraint
employee_id	INT	PK
employee_name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
phone	VARCHAR(15)	UNIQUE
hire_date	DATE	NOT NULL
department_id	INT	FK → Department
role_id	INT	FK → Role
manager_id	INT	FK → Employee
employment_status	VARCHAR(20)	NOT NULL

Possible employment statuses:

Active
Inactive
Onboarding
Resigned
Terminated

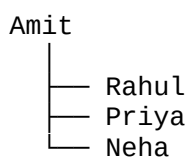
Important: manager_id

manager_id is a **self-referencing foreign key**.



This represents the reporting hierarchy.

Example:



Amit's employee_id will be stored as manager_id for Rahul, Priya, and Neha.

3.4 LEAVE_TYPE

Defines the types of leave available in the organization.

Column	Data Type	Key / Constraint
leave_type_id	INT	PK
leave_type_name	VARCHAR(50)	NOT NULL, UNIQUE
description	VARCHAR(255)	Optional
max_days_per_year	INT	CHECK > 0
requires_approval	BOOLEAN	NOT NULL, DEFAULT TRUE

Example

- 1 → Casual Leave
- 2 → Sick Leave
- 3 → Earned Leave
- 4 → Unpaid Leave

3.5 LEAVE_BALANCE

Stores the leave entitlement and usage of an employee.

Column	Data Type	Key / Constraint
balance_id	INT	PK
employee_id	INT	FK → Employee
leave_type_id	INT	FK → Leave Type
leave_year	INT	NOT NULL
allocated_days	DECIMAL(5,2)	NOT NULL, CHECK >= 0
used_days	DECIMAL(5,2)	NOT NULL, CHECK >= 0
remaining_days	DECIMAL(5,2)	NOT NULL, CHECK >= 0

Important unique constraint

UNIQUE(employee_id, leave_type_id, leave_year)

This means an employee can have only **one balance record for a particular leave type in a particular year**.

Example:

Employee	Leave Type	Year	Allocated	Used	Remaining
Rahul	Casual Leave	2026	12	4	8
Rahul	Sick Leave	2026	10	2	8
Priya	Casual Leave	2026	12	6	6

Balance calculation

remaining_days = allocated_days - used_days

3.6 LEAVE_REQUEST

Stores employee leave applications.

Column	Data Type	Key / Constraint
leave_request_id	INT	PK
employee_id	INT	FK → Employee
leave_type_id	INT	FK → Leave Type
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
total_days	DECIMAL(5,2)	NOT NULL, CHECK > 0
reason	TEXT	NOT NULL

Column	Data Type	Key / Constraint
status	VARCHAR(20)	NOT NULL
requested_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
cancelled_at	TIMESTAMP	Optional

Status

Pending
Approved
Rejected
Cancelled

Important constraint

end_date >= start_date

3.7 APPROVAL_HISTORY

Stores every approval-related action performed on a leave request.

Column	Data Type	Key / Constraint
approval_history_id	INT	PK
leave_request_id	INT	FK → Leave Request
approver_id	INT	FK → Employee
action	VARCHAR(20)	NOT NULL
comments	TEXT	Optional
action_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Possible actions:

Approved
Rejected
Cancelled

4. ER Diagram

In the pdf.

erDiagram

```

DEPARTMENT {
    INT department_id PK
    VARCHAR department_name UK
    VARCHAR description
}

```

```
ROLE {  
    INT role_id PK  
    VARCHAR role_name UK  
    VARCHAR description  
}
```

```
EMPLOYEE {  
    INT employee_id PK  
    VARCHAR employee_name  
    VARCHAR email UK  
    VARCHAR phone UK  
    DATE hire_date  
    INT department_id FK  
    INT role_id FK  
    INT manager_id FK  
    VARCHAR employment_status  
}
```

```
LEAVE_TYPE {  
    INT leave_type_id PK  
    VARCHAR leave_type_name UK  
    VARCHAR description  
    INT max_days_per_year  
    BOOLEAN requires_approval  
}
```

```
LEAVE_BALANCE {  
    INT balance_id PK  
    INT employee_id FK  
    INT leave_type_id FK  
    INT leave_year  
    DECIMAL allocated_days
```

```
    DECIMAL used_days
    DECIMAL remaining_days
}
```

```
LEAVE_REQUEST {
    INT leave_request_id PK
    INT employee_id FK
    INT leave_type_id FK
    DATE start_date
    DATE end_date
    DECIMAL total_days
    TEXT reason
    VARCHAR status
    TIMESTAMP requested_at
    TIMESTAMP cancelled_at
}
```

```
APPROVAL_HISTORY {
    INT approval_history_id PK
    INT leave_request_id FK
    INT approver_id FK
    VARCHAR action
    TEXT comments
    TIMESTAMP action_at
}
```

```
DEPARTMENT ||--o{ EMPLOYEE : has
ROLE ||--o{ EMPLOYEE : assigned_to
EMPLOYEE ||--o{ EMPLOYEE : manages
EMPLOYEE ||--o{ LEAVE_BALANCE : has
LEAVE_TYPE ||--o{ LEAVE_BALANCE : defines
EMPLOYEE ||--o{ LEAVE_REQUEST : submits
```

LEAVE_TYPE ||--o{ LEAVE_REQUEST : used_for
 LEAVE_REQUEST ||--o{ APPROVAL_HISTORY : has
 EMPLOYEE ||--o{ APPROVAL_HISTORY : performs

5. Relationship Explanation

Relationship	Cardinality	Explanation
Department → Employee	1:M	One department can have many employees
Role → Employee	1:M	One role can be assigned to many employees
Employee → Employee	1:M	One manager can manage many employees
Employee → Leave Balance	1:M	One employee can have multiple leave-type balances
Leave Type → Leave Balance	1:M	One leave type can belong to many employee balances
Employee → Leave Request	1:M	One employee can submit many leave requests
Leave Type → Leave Request	1:M	One leave type can be used by many requests
Leave Request → Approval History	1:M	One request can have multiple approval actions
Employee → Approval History	1:M	An authorized employee can perform many approval actions

6. Primary Keys

Every entity has a primary key:

DEPARTMENT
 department_id

ROLE
 role_id

EMPLOYEE
 employee_id

LEAVE_TYPE
 leave_type_id

LEAVE_BALANCE
 balance_id

LEAVE_REQUEST
 leave_request_id

APPROVAL_HISTORY
 approval_history_id

Primary keys uniquely identify records.

7. Foreign Keys

Employee

department_id
→ Department.department_id

role_id
→ Role.role_id

manager_id
→ Employee.employee_id

Leave Balance

employee_id
→ Employee.employee_id

leave_type_id
→ Leave_Type.leave_type_id

Leave Request

employee_id
→ Employee.employee_id

leave_type_id
→ Leave_Type.leave_type_id

Approval History

leave_request_id
→ Leave_Request.leave_request_id

approver_id
→ Employee.employee_id

8. Required Fields

Important fields should not be allowed to contain NULL.

Employee

employee_name
email
hire_date
department_id
role_id
employment_status

Leave Type

leave_type_name
max_days_per_year
requires_approval

Leave Balance

employee_id
leave_type_id
leave_year
allocated_days
used_days
remaining_days

Leave Request

employee_id
leave_type_id
start_date
end_date
total_days
reason
status
requested_at

Approval History

leave_request_id
approver_id
action
action_at

9. Unique Fields

These should be unique:

Department.department_name
Role.role_name
Employee.email
Leave_Type.leave_type_name

10. Important Database Constraints

Employee Email

UNIQUE(email)

Two employees cannot have the same email.

Leave Dates

CHECK(end_date >= start_date)

Leave Days

```
CHECK(total_days > 0)
```

Allocated Days

```
CHECK(allocated_days >= 0)
```

Used Days

```
CHECK(used_days >= 0)
```

Remaining Days

```
CHECK(remaining_days >= 0)
```

Leave Status

```
CHECK(status IN  
( 'Pending', 'Approved', 'Rejected', 'Cancelled' ))
```

Employment Status

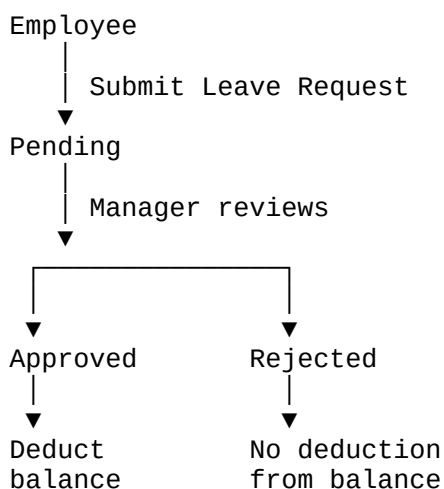
```
CHECK(employment_status IN  
( 'Active', 'Inactive', 'Onboarding', 'Resigned', 'Terminated' ))
```

Approval Action

```
CHECK(action IN  
( 'Approved', 'Rejected', 'Cancelled' ))
```

11. Real-World RBAC Approval Flow

The actual workflow should be:



An employee **cannot approve or reject their own request**.

The system should verify:

`approver_id != leave_request.employee_id`

and:

`approver has approval permission`

and, in a manager-based system:

`approver_id = employee.manager_id`

for the employee whose request is being reviewed.

12. Important Design Decision: Why We Need Role

Without a **ROLE** table, the database knows that rohit is an employee, but it doesn't properly represent **what rohit is authorized to do**.

With:

EMPLOYEE
↓
ROLE

we can represent:

Rahul → Employee
rohit → Manager
Neha → HR
Admin → Administrator

This makes the system much closer to a real-world application.

Explanation of Design Decisions

1. Department and Role are separate tables

Department and Role are stored separately to avoid repeating the same department and role information for every employee.

2. Employee stores department and role references

`department_id` and `role_id` are foreign keys in the Employee table. This connects each employee to their department and assigned role.

3. Manager is represented as an Employee

A separate Manager table is not required because a manager is also an employee. The `manager_id` field references `employee_id` in the same Employee table.

4. Self-referencing relationship for managers

`manager_id` allows one employee to manage multiple employees while each employee can have one assigned manager.

5. Leave Type is separated from Leave Request

Leave types such as Casual, Sick, and Earned Leave are stored separately so that their rules can be managed without repeating information in every leave request.

6. Leave Balance tracks yearly entitlement

The Leave Balance table stores allocated, used, and remaining leave for each employee and leave type for a particular year.

7. Unique leave balance per year

`UNIQUE (employee_id, leave_type_id, leave_year)` prevents duplicate leave-balance records for the same employee, leave type, and year.

8. Leave Request stores employee leave applications

The Leave Request table records when an employee applies for leave, including dates, leave type, reason, total days, and current status.

9. Approval History maintains an audit trail

Approval History records approval, rejection, or cancellation actions along with the person who performed the action and the time of the action.

10. Primary and foreign keys maintain data integrity

Primary keys uniquely identify records, while foreign keys ensure that related records refer to existing data.

11. Constraints enforce business rules at database level

`NOT NULL`, `UNIQUE`, and `CHECK` constraints prevent invalid or incomplete data from being stored.

12. Status fields use CHECK constraints

Employee and leave-request statuses are restricted to predefined values such as `Active`, `Pending`, `Approved`, `Rejected`, and `Cancelled`.

13. Dates are validated

The constraint `end_date >= start_date` prevents a leave request from having an invalid date range.

14. Identity columns are used for IDs

`GENERATED ALWAYS AS IDENTITY` automatically generates unique IDs, reducing manual ID management.

15. The design supports future expansion

The structure can be extended later with features such as multiple approval levels, holiday calendars, attendance integration, notifications, and leave policies without redesigning the entire database.

Business Rules – Employee Leave Management System

1. **Employee cannot request leave without sufficient leave balance.**
The requested leave days must not exceed the employee's available balance.
2. **Only active employees can request leave.**
Employees with `Inactive`, `Resigned`, or `Terminated` status cannot submit a leave request.
3. **Leave type must be valid.**
An employee can request only a leave type that exists in the `LEAVE_TYPE` table.
4. **Leave dates cannot overlap.**
An employee cannot have two leave requests covering overlapping dates.
5. **End date cannot be before start date.**
`end_date` must always be greater than or equal to `start_date`.
6. **Total leave days must be greater than zero.**
A leave request cannot contain zero or negative leave days.
7. **Manager cannot approve their own leave.**
The employee submitting the request cannot be the employee performing the approval.
8. **Only an authorized manager can approve or reject leave.**
The approver must have the required role/permission to perform the action.
9. **A manager should approve leave for their assigned employees.**
The approver should match the employee's `manager_id` where manager-based approval is used.
10. **Rejected leave should not reduce the leave balance.**
When a request is rejected, `used_days` and `remaining_days` must remain unchanged.
11. **Approved leave should reduce the leave balance.**
When leave is approved, the approved number of days should be added to `used_days` and deducted from `remaining_days`.
12. **Cancelled leave should restore the balance where applicable.**
If an already-approved leave is cancelled, the previously deducted days should be returned to the employee's balance.
13. **Pending leave should not permanently reduce the balance.**
A pending request should not be counted as used leave until it is approved.

14. Every approval or rejection should be recorded in Approval History.

The system must maintain who performed the action, what action was taken, and when it occurred.

15. An employee cannot have duplicate leave balances for the same year and leave type.

Each employee can have only one balance record for a particular leave type and year.

16. Leave allocation cannot be negative.

`allocated_days`, `used_days`, and `remaining_days` cannot contain negative values.

17. Remaining leave cannot exceed the allocated leave.

Normally, $\text{remaining_days} = \text{allocated_days} - \text{used_days}$.

18. Leave requests must have a reason.

An employee must provide a reason when submitting a leave request.

19. Leave request status must be valid.

A request can only have predefined statuses such as Pending, Approved, Rejected, or Cancelled.

20. A cancelled request must have a cancellation timestamp.

When the status is Cancelled, `cancelled_at` must contain the cancellation time.